

Deploy the Reports backend (Apps Script)

The Reports page reads your **private** weekly SM-performance Google Sheet. Because GitHub Pages is a static site, it cannot hold Google credentials — so a small Apps Script web app does the reading and returns only the parsed **numeric** series as JSON. (Qualitative rows like Learnings / Remarks are never returned.)

✓ Why we host this in the TargetSetter sheet (important)

The weekly SM Performance sheet lives under the **sadhguru.org Workspace**, whose admin **blocks** Apps Script web-app deployment — that's the “not authorised to deploy” error. The **TargetSetter Data** sheet is owned by an account that **can** deploy.

A script can read **any** sheet the executing account has access to. So instead of a second deployment, the Reports logic now lives **inside the TargetSetter Apps Script project** (`Code.gs`) and is served from the **same** `/exec` URL:

Request	Returns
<code>.../exec</code>	TargetSetter data (Brands + MonthlyPlan)
<code>.../exec?mode=reports</code>	Weekly SM Performance parsed series
<code>POST .../exec</code>	Overwrite TargetSetter data

Requirement: the account that deploys/runs the script (Execute as: **Me**) must have at least **Viewer** access to the weekly sheet (`abhilash.jain@sadhguru.org` already does).

⚠ **Privacy note.** The web app runs with Who has access: Anyone, and its `/exec` URL is in the public repo. The **sheets stay private**, but anyone with the URL could fetch the numeric report data. If that's not acceptable, set `REPORTS_SCRIPT_URL = ""` to keep the Reports page in **Demo mode**.

Steps

1. Open the **TargetSetter Data** sheet:
https://docs.google.com/spreadsheets/d/1EuvPfL6b-XvNYRuuGWr5Ab_Bk5Pbz4H8EtK5YdTBqn0/edit
2. Menu: **Extensions ▶ Apps Script** (this is the project you already deploy).
3. Select all in `Code.gs` and replace it with the entire contents of the `Code.gs` in this folder (the new combined version).
4. Click **Save** (💾).
5. **Deploy ▶ Manage deployments ▶** click the  (edit) on the existing deployment ▶ **Version: New version ▶ Deploy**.
- This keeps the **same** `/exec` URL, so the TargetSetter dashboard and the

pre-filled `reports-config.js` keep working with no further changes.

- (First time only, if there's no deployment yet: **Deploy ► New deployment ► Web app ► Execute as: Me ► Who has access: Anyone.**)

6. If prompted, **Authorize** — review the permissions and Allow. (The script now also reads the weekly sheet, so it may ask again.)

That's it. `reports-config.js` is already set to:

```
const REPORTS_SCRIPT_URL = "https://script.google.com/macros/s/AKfycbz4rTiXojBGsH9r-bItTurQaivj69EtJyxaqYch0vVaQsxXUFElwsH8A16uTovMH476utA/exec?mode=reports";
```

If your `/exec` URL ever changes, update both `config.js` (TargetSetter) and `reports-config.js` (same URL + `?mode=reports`).

Quick test

Open `.../exec?mode=reports` directly in your browser. You should see JSON like:

```
{ "generatedAt": "...", "brands": { "Isha Foundation": { "weeks": [...], "platforms": {...} }, ... } }
```

Then open the Reports page and click  **Refresh** — the badge flips from **Demo** to **Live** and a spinner shows while data is pulled.

Safety

Until the combined script is deployed, the Reports page automatically stays in **Demo mode** (it validates the response shape and falls back to the bundled snapshot if the endpoint doesn't return real reports data) — so the live site won't break in between.